

Question Bank — Week 12

Heaps and Priority Queues (Lecture 12) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the code	4
Part D — Heap state — draw every step	6
Part E — Complexity analysis	7
Part F — Find and fix the bug	8
Part G — Write the code	8

Questions: [week12-questions.md](#). Commit to your own answer before reading one here. Every array, trace and count below was produced by running the reference solution, `solutions/dsa/heap.py`.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	b	M19	d
M02	c	M08	d	M14	c	M20	c
M03	a	M09	c	M15	d	M21	a
M04	d	M10	a	M16	b	M22	d
M05	a	M11	b	M17	c		
M06	c	M12	d	M18	a		

W12-M01 — b. That is the heap property, and it is all a heap promises. The array is not sorted (a): Q itself is `[1, 5, 3, ...]`. Siblings are unordered (c). The largest value is in some leaf, not necessarily the last one (d).

W12-M02 — c. $2i + 1$; the right child is $2i + 2$ (d). $2i$ (a) is the 1-based formula of CLRS.

W12-M03 — a. $(10 - 1) // 2 = 4$. Check: the children of 4 are 9 and 10.

W12-M04 — d. $\lfloor \log_2 1000 \rfloor = 9$: levels 0 to 9 hold up to 1,023 nodes. 10 (a) is the number of **levels**.

W12-M05 — a. 3's children are 7 and 4; 2 has none. In (b), 2 at index 3 is below its parent 3; in (c), 1 is below 2; in (d), 3 at index 3 is below 4.

W12-M06 — c. 2 is appended at index 9, under 6 (index 4): swap. Now at index 4, under 5 (index 1): swap. Under 1 at the root: stop. The array becomes [1, 2, 3, 8, 5, 4, 9, 11, 10, 6].

W12-M07 — b. 10, the last item, moves to the root; its children are 5 and 3, so it swaps with 3, then with 4. The array is [3, 5, 4, 8, 6, 10, 9, 11]. The new root is always the smaller of the old root's two children.

W12-M08 — d. Swapping with a child that beats the item but not its sibling puts a larger value above a smaller one (W12-B2). (b) is wrong: sift-down never changes the shape.

W12-M09 — c. $n // 2 - 1$ is the last index with a child; everything after it is a leaf, already a heap of one.

W12-M10 — a. At most about n swaps and $2n$ comparisons (W12-E3).

W12-M11 — b. Half the nodes are leaves (no work), a quarter move at most one level, and so on: $\sum h \cdot n / 2^{h+1} \leq n$. It does look at every value (a) — it just does little with most of them.

W12-M12 — d. Each new value is the smallest so far and climbs to the root: about $\log_2 i$ swaps for the i -th push, $\Theta(n \log n)$ in total. On random input the pushes average about two comparisons each.

W12-M13 — b. The best value is at index 0.

W12-M14 — c. The heap property says nothing about where a value that is not the best might be — it could be in any subtree. Only the root is cheap to find.

W12-M15 — d. Complete shape means indices $0 \dots n - 1$ with no holes, and $2i + 1$, $2i + 2$, $(i - 1) // 2$ replace left, right and parent.

W12-M16 — b. b and c both have priority 1; b was enqueued first (counter 1 against 2), so it is served first. Then c, then a.

W12-M17 — c. With $(1, \text{item_b})$ against $(1, \text{item_c})$, Python compares the items: dictionaries raise `TypeError`, strings silently order alphabetically. The unique counter decides every tie before the item is reached.

W12-M18 — a. n values, each costing at most one peek and a pop + push on a heap of k : $O(n \log k)$. (c) is sorting everything.

W12-M19 — d. The root is the smallest of the k kept, and it is the one a new, larger value must replace.

W12-M20 — c. Heap sort builds the heap inside the input array and moves each maximum into the space the heap gives up.

W12-M21 — a. The sorted region grows from the right as the heap shrinks from the right. Heap sort is not stable (b).

W12-M22 — d. heapify gives [9, 6, 4, 1, 5, 3, 2, 1]: `_items[1]` is 6.

Part B — Short answer and essay

W12-E1 (4)

- **Shape:** a **complete** binary tree — every level full except possibly the last, which is filled from the left.
- **Order (heap property):** every parent beats its children — $\leq$ in a min-heap, $\geq$ in a max-heap. So the best value is at the root. Siblings are unordered.
- **Height:** a complete tree of n nodes has height $\lfloor \log_2 n \rfloor$, so every operation that walks one path is $O(\log n)$. A heap can never degenerate into a list, unlike an unbalanced BST.
- **Storage:** level-by-level numbering fills indices $0 \dots n - 1$ with no gaps, so the tree is an array; children at $2i + 1$ and $2i + 2$, parent at $(i - 1) // 2$. No nodes, no references.

W12-E2 (4)

- **push(x):** append x at index n (the only place that keeps the shape complete), then **sift up**: while x beats its parent, swap them. At most one swap per level: $O(\log n)$.
- **pop():** raise `IndexError` if empty. Save the root; remove the **last** item; if the heap is now empty, return it (the one-item case); otherwise put it at the root and **sift down**: find the better of the item and its (up to two) children; if a child is better, swap and continue. At most two comparisons per level: $O(\log n)$.
- **Special cases:** empty pop/peek; a single item; a node with a left child but no right child (check each child index against `len` before reading it).
- peek is $O(1)$: the root.

W12-E3 (4)

- **Algorithm:** leave the values in place; for i from $n // 2 - 1$ **down** to 0, sift down i . Leaves (the back half) are heaps already.
- **Why backwards:** when i is sifted down, both of its subtrees must already be heaps — sift-down only repairs a root sitting on two valid heaps. Going backwards guarantees that; going forwards does not (W12-K3).
- **O(n):** a node of height h moves at most h levels, and at most $\lceil n/2^{h+1} \rceil$ nodes have height h , so the work is at most $\sum_h h \cdot n/2^{h+1} = \frac{n}{2} \sum_h h/2^h = \frac{n}{2} \cdot 2 = n$ swaps.
- **n pushes:** a push costs the node's **depth**, and half the nodes are at the bottom, at depth $\approx \log_2 n$: in the worst case (descending input for a min-heap) the total is $\Theta(n \log n)$. Measured in Lecture 12: 2.0 comparisons per element for heapify against 14 per element for pushes, at $n = 2^{16}$.

W12-E4 (3)

	enqueue	dequeue	peek
unsorted array	$O(1)$ amortised	$O(n)$	$O(n)$
sorted array	$O(n)$	$O(1)$ (from the end)	$O(1)$
binary heap	$O(\log n)$	$O(\log n)$	$O(1)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$, or $O(1)$ with a pointer to the minimum

- **Choose the heap:** a workload of n enqueues and n dequeues costs $O(n \log n)$, against $O(n^2)$ for either array; it lives in one array with no references, and needs no balancing. The BST matches the Big-O but costs nodes, references and a balancing scheme — worth it only if you also need to search for or delete **arbitrary** items.

W12-E5 (3)

- With `(priority, item)`, two equal priorities make Python compare the items. Items that do not support `<` (dictionaries, most objects) raise `TypeError`; items that do (strings) are ordered by value, which is not what anyone asked for.
- The counter is unique and sits before the item, so every comparison is settled by priority or counter and never reaches the item.
- It also makes ties **FIFO**: the counter grows with each enqueue, so of two equal priorities the earlier one is served first. A heap on its own is not stable, so this fairness comes only from the counter.

Part C — Trace the code

W12-T1

Operation	Array afterwards	Returns
push 6	[6]	
push 2	[2, 6] — 2 swaps with 6	
push 8	[2, 6, 8]	
push 1	[1, 2, 8, 6] — index 3 → 1 → 0	
push 5	[1, 2, 8, 6, 5] — 5 is not below 2	
push 3	[1, 2, 3, 6, 5, 8] — 3 swaps with 8	
pop	[2, 5, 3, 6, 8] — 8 to the root, then swaps with 2, then with 5	1
pop	[3, 5, 8, 6] — 8 to the root, swaps with 3	2

W12-T2. $n = 10$, so the last parent is $10 // 2 - 1 = 4$.

sift_down(i)	Value	Moves	Array afterwards
4	4	swaps with 0 (index 9)	[3, 9, 2, 1, 0, 5, 8, 7, 6, 4]
3	1	stays (children 7, 6)	unchanged
2	2	stays (children 5, 8)	unchanged
1	9	swaps with 0 (index 4), then with 4 (index 9)	[3, 0, 2, 1, 4, 5, 8, 7, 6, 9]
0	3	swaps with 0 (index 1), then with 1 (index 3)	[0, 1, 2, 3, 4, 5, 8, 7, 6, 9]

5 swaps (and 14 comparisons) in all. The result is a valid min-heap; it happens to look almost sorted, but 8 before 7 and 6 shows it is not.

W12-T3

Pop	Returns	Array afterwards	How
1	1	[3, 5, 4, 8, 6, 10, 9, 11]	10 to the root; swaps with 3, then 4
2	3	[4, 5, 9, 8, 6, 10, 11]	11 to the root; swaps with 4, then 9
3	4	[5, 6, 9, 8, 11, 10]	11 to the root; swaps with 5, then 6

Each pop returns the next smallest value: 1, 3, 4.

W12-T4

Operation	Heap array afterwards	Returns
enqueue report, 3	(3,0,report)	
enqueue login, 1	(1,1,login)	
	(3,0,report)	
enqueue email, 2	(1,1,login)	
	(3,0,report)	
	(2,2,email)	
dequeue	(2,2,email)	fix login bug
	(3,0,report)	
enqueue server, 1	(1,3,server)	
	(3,0,report)	
	(2,2,email)	

Operation	Heap array afterwards	Returns
enqueue docs, 3	(1,3,server) (3,0,report) (2,2,email) (3,4,docs)	
dequeue	(2,2,email) (3,0,report) (3,4,docs)	restart server
dequeue	(3,0,report) (3,4,docs)	reply to email
dequeue	(3,4,docs)	print report

“print report” and “update docs” tie at priority 3; the report was enqueued first (counter 0 against 4) and is served first.

W12-T5. heap_sort_steps in solutions/dsa/sorting.py:

Stage	Array	
input	[4, 10, 3, 5, 1]	
build (max-heap)	[10, 5, 3, 4, 1]	sift_down(1): 10 stays; sift_down(0): 4 swaps with 10, then with 5
swap 0 ↔ 4, sift down in 0..3	[5, 4, 3, 1, 10]	
swap 0 ↔ 3, sift down in 0..2	[4, 1, 3, 5, 10]	
swap 0 ↔ 2, sift down in 0..1	[3, 1, 4, 5, 10]	
swap 0 ↔ 1	[1, 3, 4, 5, 10]	sorted

Part D — Heap state — draw every step

W12-S1 (max-heap; draw each array as a tree, children of i at $2i + 1$ and $2i + 2$)

Push	Array afterwards	Swaps
5	[5]	—
12	[12, 5]	12 ↔ 5
7	[12, 5, 7]	none: 7 < 12
20	[20, 12, 7, 5]	20 ↔ 5 (index 3 → 1), 20 ↔ 12 (1 → 0)
3	[20, 12, 7, 5, 3]	none: 3 < 12

Push	Array afterwards	Swaps
15	[20, 12, 15, 5, 3, 7]	$15 \leftrightarrow 7$ (index 5 $\rightarrow$ 2); $15 < 20$, stop

Final tree: 20 at the root; children 12 and 15; 12's children 5 and 3; 15's child 7.

W12-S2 ($k = 3$)

Value	Action	Heap array afterwards
5	push (fewer than 3)	[5]
1	push	[1, 5]
9	push	[1, 5, 9]
3	$3 > \text{root } 1$: pop 1, push 3	[3, 9, 5]
7	$7 > \text{root } 3$: pop 3, push 7	[5, 9, 7]
2	$2 \leq \text{root } 5$: skip	[5, 9, 7]
8	$8 > \text{root } 5$: pop 5, push 8	[7, 9, 8]

The heap holds 7, 8, 9: the 3 largest. Popping them gives 7, 8, 9, so `top_k` reverses that to [9, 8, 7].

W12-S3

- (a) [1, 2, 3, 4, 5, 6] — **valid** (a sorted array is always a min-heap; the reverse is false).
- (b) [1, 3, 2, 5, 4, 1] — **not valid**: 1 at index 5 is below its parent 2 (index 2). Only that pair; 1 at the root and 1 at index 5 are not parent and child.
- (c) [2, 2, 2, 3, 2] — **valid**: equal values do not break $\leq$.
- (d) [4, 5, 6, 7, 8, 9, 3] — **not valid**: 3 at index 6 is below 6 (index 2). 3 is also smaller than the root, but the property only compares parents with children; the violation is the pair (6, 3).

Part E — Complexity analysis

W12-K1 — $\Theta(n \log n)$ **in the worst case**. n pushes cost up to $\log_2 i$ each, and n pops up to $2 \log_2 i$ comparisons each. result is values **sorted ascending**: this is heap sort, written with a priority queue. Against Week 10's `heap_sort`: the same $O(n \log n)$ time (with a worse constant, and the build is $O(n \log n)$ instead of `heapify`'s $O(n)$), but $O(n)$ **extra space** — the heap and the result list — where the in-place version needs $O(1)$.

W12-K2 — $O(n \log k)$ **time**, $O(k)$ **space**, against $O(n \log n)$ time and $O(n)$ space for sorting. For $k = 10$, $n = 10^6$: each value costs one peek comparison, plus a pop and a push on a heap of height 3 when it gets in — a few million comparisons at most, against about 2×10^7 for the sort — and the heap needs 10 slots, so it also works on a stream too large

for memory. For $k = n$ the two are the same, $O(n \log n)$: the heap is then a whole heap sort.

W12-K3. The forward `sift_down` loop is **wrong**. On `[5, 4, 3, 2, 1]` it gives `[3, 1, 5, 2, 4]` — 1 is below 3. On the lecture's `[9, 4, 7, 1, 8, 2, 6, 3, 5]` it gives `[4, 1, 2, 3, 8, 7, 6, 9, 5]`. Sifting down the root first compares it with children that are not yet heaps, so smaller values further down are never brought up to the root. (It happens to work on some inputs, such as `[3, 2, 1]`.)

A forward loop of `self._sift_up(index)` for every index from 1 to $n - 1$ **is** correct: when index i sifts up, indices $0 \dots i - 1$ already form a heap, so it is exactly n pushes done in place. It costs $\Theta(n \log n)$ in the worst case — correct, but it misses the point of `heapify`.

Part F — Find and fix the bug

W12-B1. `parent = index // 2` is the **1-based** formula. With 0-based indices the parent of i is `(index - 1) // 2`; `index // 2` is right only for odd i , and for even i it names the node **after** the true parent — for index 4, node 2 instead of node 1. The sift-up then compares with the wrong node. `MinHeap([1, 5, 2, 6]).push(3)` appends 3 at index 4, compares it with 2 (index 2), stops, and leaves `[1, 5, 2, 6, 3]` — 3 below 5, an invalid heap. The correct result is `[1, 3, 2, 6, 5]`. **Fix:** `parent = (index - 1) // 2`.

W12-B2. It swaps with the **first** child that beats the item, not the **better** child. When both children beat it and the right is better, the left moves up above the right. Popping `[2, 4, 3, 9, 7, 8, 5, 12, 10]` returns 2 but leaves `[4, 9, 3, 10, 7, 8, 5, 12]` — 4 above 3 — and the next pop returns 4 instead of 3. **Fix:** find the better child first — start `best = index`, replace it by the left child if the left beats `items[best]`, then by the right child if the right beats `items[best]`; stop if `best == index`, otherwise swap with `best`.

W12-B3. No one-item case. When the heap holds a single item, `pop()` on the array empties it, and `self._items[0]` raises `IndexError` — on a heap that was **not** empty. `MinHeap([4, 7])`: the first pop returns 4, the second raises. **Fix:** after removing the last item, `if len(self._items) == 0: return last`.

W12-B4. The range stops **before** 0, so the root is never sifted down. `MinHeap([9, 4, 7, 1, 8, 2, 6, 3, 5])` ends as `[9, 1, 2, 3, 8, 7, 6, 4, 5]` and `peek()` returns 9, the **largest** value. Every subtree below the root is a heap, which is why the result looks nearly right. **Fix:** `range(len(self._items) // 2 - 1, -1, -1)` — a stop of `-1` includes 0.

Part G — Write the code

W12-C1

```
def is_min_heap(values):
    for child in range(1, len(values)):
        if values[child] < values[(child - 1) // 2]:
            return False
    return True
```

Every element except the root has exactly one parent, so checking each **child** against its parent checks every edge of the tree once: $n - 1$ comparisons, $O(n)$, and no heap object at all. Starting at index 1 avoids the root, which has no parent. Walking the **parents** instead needs two children per step and two bounds checks — correct, but longer. This is also the body of `MinHeap.is_valid`, with `<` in place of `_beats`.

W12-C2

```
def top_k(values, k):
    if k < 0:
        raise ValueError("k must be >= 0")
    if k == 0:
        return []
    heap = MinHeap() # the best k so far; root = weakest
    for value in values:
        if len(heap) < k:
            heap.push(value)
        elif heap.peek() < value: # beats the weakest: replace it
            heap.pop()
            heap.push(value)
    result = [heap.pop() for _ in range(len(heap))] # smallest first
    result.reverse()
    return result
```

The heap holds the best k values seen so far, and its root is the **weakest** of them — exactly the value a better newcomer must replace. A value that does not beat the root costs one comparison and is forgotten. Each value costs at most $O(\log k)$, so the total is $O(n \log k)$ in time and $O(k)$ in space. Popping the heap at the end gives the k values smallest first; one reverse makes them largest first. A max-heap of all n values would also work, but it is $O(n)$ space and $O(n + k \log n)$ time, and it cannot run on a stream.

W12-C3

```
def merge_sorted(lists):
    heap = MinHeap()
    for which, values in enumerate(lists):
        if values:
            heap.push((values[0], which, 0)) # (value, list, position)
    merged = []
    while not heap.is_empty():
        value, which, position = heap.pop()
        merged.append(value)
```

```

    if position + 1 < len(lists[which]):
        heap.push((lists[which][position + 1], which, position + 1))
    return merged

```

Each heap entry is (value, list number, position): the current front of one list, and where it came from, so that after popping it the **next** value of the same list can be pushed. The heap never holds more than k entries — one per list — so every pop and push is $O(\log k)$, and N values cost $O(N \log k)$. The list number also breaks ties between equal values, so the positions are never compared with anything but integers. The input lists are only read. This is the merge step of an external sort, which merges k sorted runs of a file too large for memory.

W12-C4

```

def join_ropes(lengths):
    heap = MinHeap(lengths)                # heapify: O(n)
    total = 0
    while len(heap) > 1:
        joined = heap.pop() + heap.pop()    # the two shortest
        total += joined
        heap.push(joined)
    return total

```

Why the two shortest: every rope's length is paid once for each join it takes part in, so a rope that is joined early is paid for many times. The shortest ropes should therefore be the ones buried deepest — joined first. Formally this is Huffman's argument (1952): in an optimal plan the two shortest ropes can always be joined together first, and the rest of the problem is the same problem with one rope fewer. `MinHeap(lengths)` heapifies a **copy** in $O(n)$; then $n - 1$ rounds of two pops and a push, $O(n \log n)$. Re-sorting a list after every join would be $O(n^2 \log n)$.

W12-C5

```

def running_medians(values):
    low = MaxHeap()                        # smaller half; root = its largest
    high = MinHeap()                       # larger half; root = its smallest
    medians = []
    for value in values:
        if low.is_empty() or value <= low.peek():
            low.push(value)
        else:
            high.push(value)
        if len(low) > len(high) + 1: # rebalance: sizes differ by <= 1
            high.push(low.pop())
        elif len(high) > len(low):
            low.push(high.pop())
        if len(low) > len(high):
            medians.append(low.peek())

```

```
    else:
        medians.append((low.peek() + high.peek()) / 2)
    return medians
```

low holds the smaller half as a **max**-heap, so its root is the largest of the small values; high holds the larger half as a **min**-heap, so its root is the smallest of the large values. Every value in low is $\leq$ every value in high, and the sizes differ by at most one, with low never the smaller. So the two roots are the two middle values of everything seen: with an odd count, the median is low's root; with an even count, the mean of the two roots. Each new value costs at most three pushes or pops: $O(\log n)$, and $O(n \log n)$ for the whole list — against $O(n^2)$ for keeping a sorted list and inserting into it.